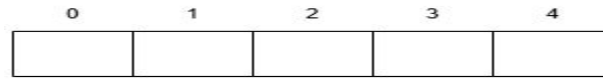


Lecture 15

Queue

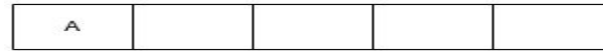
- A queue is a **linear data** structure in which insertions(also called additions and **enqueue**) and deletions(also called removals and **dequeue**) take place at different ends.
- The end at which new elements are added is called **rear**, and the end at which old elements are deleted is called **front**.
- The restrictions on a queue imply that if we insert A, B, C, D and E in the order, then A is the first element deleted from the queue.
- Since the first element inserted into a queue is the first element removed, queues are also known as **First-In-First-Out** (FIFO).

Inserting and deleting in a queue



f r

enqueue('A')



f

r

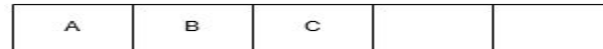
enqueue('B')



f

r

enqueue('C')



f

r

enqueue('D')



f

r

f = queue front
r = queue rear

continue...

f

r

enqueue('D')



f

r

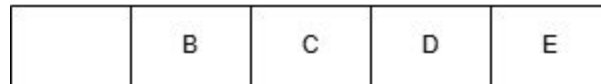
enqueue('E')



f

r

dequeue()



f

r

Queue creation

```
#define MAX_QUEUE_SIZE 5

typedef struct{
    int key;
}element;

element queue[MAX_QUEUE_SIZE];

int front = 0;

int rear = 0;
```

```
#define MAX_QUEUE_SIZE 5
int queue[MAX_QUEUE_SIZE];
int front = 0;
int rear = 0;
```

Operations on Queue

- IsFull
- enqueue
- dequeue
- IsFull

IsFull

isFull():

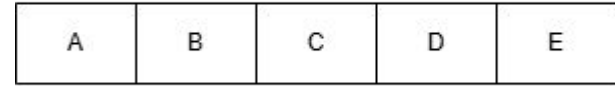
```
if(front == 0 && rear == size):
```

```
    return true;
```

```
else:
```

```
    return false;
```

enqueue('E')



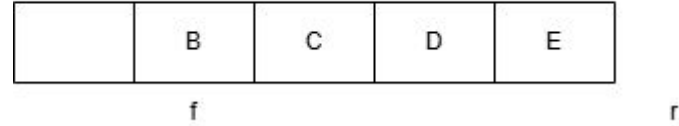
f

r

Drawback of queue

- Eventhough, we have one empty place in front of the queue. We cannot insert a new element.

enqueue('E')



Operations on Queue

- IsFull
- enqueue
- IsEmpty
- dequeue

IsFull

```
if rear == size:  
    print queue is full  
    return true  
return false
```

enqueue

enqueue element:

if rear == size:

print queue is full

return

queue[rear] = element

rear++

IsEmpty

```
if front == rear:  
    print queue is empty;  
    return true;  
return false;
```

dequeue

dequeue:

```
if front == rear:
    print queue is empty;
ans = queue[front];
queue[front] = -1;
front++;
if front == rear:
    front = 0;
    rear = 0;
return ans
```

Priority queue

- Each element in the priority queue is associated with a priority value and elements are served based on their priority rather than their insertion order.
- When a new item is added, it is inserted according to its priority.

Example

- Binary heap is the most common implementation of priority queue
- There are two types of Binary Heap i.e., Min Heap and Max Heap
- Min Heap allows quick access to the element with smallest value.
- Max Heap allows quick access to the element with largest value.